Lab3 - EEGR4613 Communications Engineering

Caleb J. Yoon

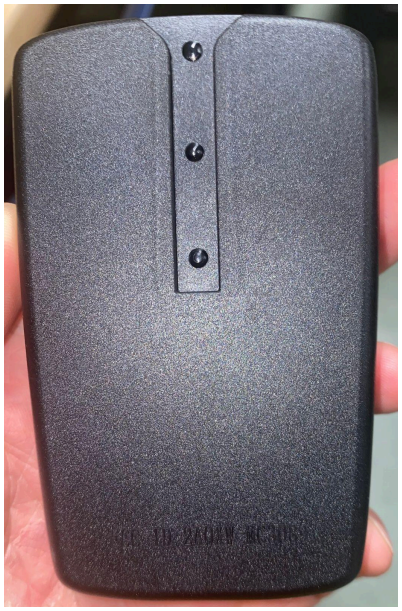
LeTourneau University
Longview TX, USA
CalebYoon@letu.edu - CalebJamesYoon@gmail.

Introduction

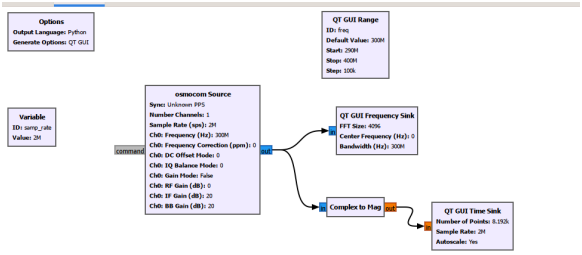
The purpose of this lab is to gain experience in using Software Defined Radios in conjunction with GNU Radio. The specific radio use was the HackRF. All the work done here has been documented on github[1].

In this lab, the objective is to decode a given garage door opener and to find which coding scheme it used. Much of the knowledge from this lab came from this tutorial[2].

The specific FCC license of the garage door opener is: 2A0XW - MC3089. According to its user manual[3] the garage door opener operates on 300Mhz.



Implementation



The flowgraph of the GNU radio diagram consists of a osmocom source and two QT GUIs. The osmocom source is the block that lets GNU radio see what signals the HackRF is capturing. If the HackRF is not connected, GNU radio will throw up an error. QT GUI frequency is used to see the frequency behavior of the signal from the garage door opener and the complex to mag converts the time signal to see the digital pulses of coding of the garage opener signal.

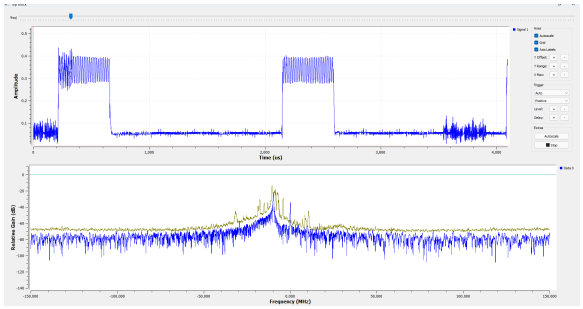
It is important to note that SDR could scan a range from 290MHz to 400MHz and step by an increment of 100kHz to be flexible and be able to capture signals from other garage door openers. The SDR had a sampling rate of 2MHz. This is the graph of a typical output of both the time and frequency graphs while the garage door opener was sending a signal. The garage opener was in fact measured around 300MHz. But due to cheap manufacturing, there was some drift with each signal, making precise frequency placement difficult.

---

1 https://github.com/CJYMage/HackRF-Garage-Door-Opener-Lab-3--Communiations-engineering.git

2 https://greatscottgadgets.com/sdr/8/.

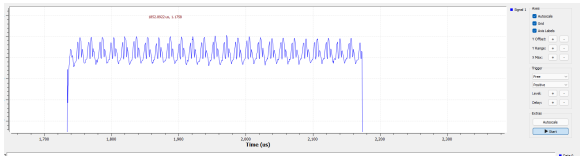
3 https://fcc.report/FCC-ID/2AQXW-MC3089/5832966.pdf



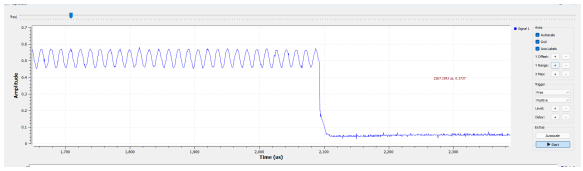
Results

Since the focus of the project was to decode the digital message of the garage door opener, the time graphs were the main focus of observation.

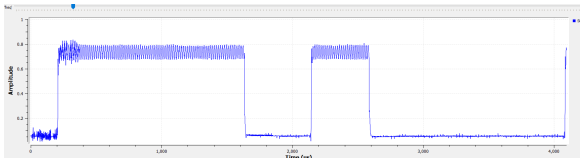
Many different signals were captured at various time resolutions. Zoomed in, it is clear to see that there is some level of noise within the signal.



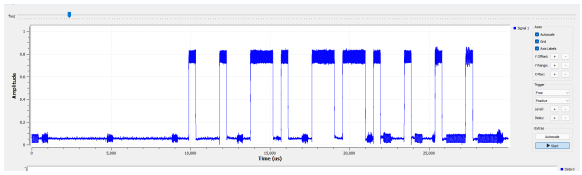
However, this noise was intermittent and a clearer signal was later captured with much less noise.



It is important to note that there were different pulse lengths. The one below shows both a long and short pulse interval.



Zoomed out, a full sequence of the digital signal can be captured. It appears that OOK (on-off keying) is being used.



From here, there are a couple of different ways to decode the signal. One way would be to see high amplitudes as 1s and zeros as 0s. But due to the difficulty it would be to figure out if a sequence started with a 1 or a 0, it is unlikely that the coding scheme is through amplitude alone.

A more likely coding scheme is that the pulse widths indicate a 1 or a 0. Assuming that the shorter width is a 1 and the longer width is a zero, then reading from left to right the presumed sequence of the above picture is 1101001111. But if it sent MSB first, then the actual sequence would be the reverse, resulting in 1111001011.

Conclusion

Overall, this lab was very illuminating. It allowed me to see that SDRs are very adaptable and can be used to capture a variety of signals. And another thing I learned was how easy it is to integrate

GNU radio and the HackRF together. It would be interesting to use this in conjunction with another tool such as wireshark. Wireshark would give me packet information of internet signals, and the SDR would give me modulation and coding information of those same signals.